PROGRAMMING 512 ASSIGNMENT

RICHFIELD
richfield.ac.za

Name and Surname: Mugadi Mpho Portia

Student ITS No: 402605278

Qualification: BSc in IT **Year of Study:** 1 **Semester:** 2

Assignment due date: 23 October 2026 **Date submitted:** 22 October 2026

QUESTION	EXAMINER MARKS	MODERATOR MARKS	REMARKS

ASSIGNMENT INSTRUCTIONS

Please tick each box to confirm completion.

Use Times New Roman font, size 12, with 1.5 line spacing throughout the document. Apply Harvard Referencing Style for all citations and references.

For essay-style assignments, please include the following sections:

Table of Contents

Introduction

Main Body (with relevant subheadings)

Conclusion

References

Submit the assignment in PDF format on Moodle. Use the specified cover page provided.

Include a signed declaration of originality.

DECLARATION OF ORIGINALITY:

I hereby declare that this assignment is my own work and has not been copied from any other source except where due acknowledgment is made. I affirm that all sources used have been properly cited and that this submission complies with the institution's policies on academic integrity and plagiarism

Student Signature: M.mp **Date:** 22/10/2026

Table of Contents

Cover page……1

Table of contents……2

Introduction 3

Coding……4 - 10

The output……11

Conclusion……12

Reference……13

Introduction

Richfield Graduate Institute of Technology has operating across multiple campuses in South Africa and relies heavily on shared academic resources such as computer laboratories, multimedia projectors, seminar rooms, and presentation equipment. These resources are essential for daily teaching, assessments, and academic events. At present, each campus manages resource bookings manually through email requests, spreadsheets, or paper-based registers.

The system provides a consistent, reliable, and user-friendly solution that supports multiple campuses, enforces booking rules, and stores data persistently for reporting and analysis. This project I created to solve that problem. The aim is to design and develop a Smart Campus Resource Booking System that allows lecturers to book resources, campus administrators to manage resources, and system operators to view reports across all campuses.

The system supports 3 user roles. A Lecturer, Campus Admin and System Operators can register, login, select a campus, book a resource, view personal booking history, and cancel a future booking. A Campus Administrator can login, add, update, and remove resources for their campus, view all bookings for their campus, and monitor usage.

A System Operator can login with full access, view booking and resource data across all campuses, and generate comparative reports. These 3 campuses were chosen because they are in different provinces and they show that each campus has its own booking rule.

```
import tkinter as tk
from tkinter import messagebox, ttk
import sqlite3

conn = sqlite3.connect("booking.db")
cur = conn.cursor()

cur.execute("CREATE TABLE IF NOT EXISTS users (username TEXT, password TEXT, role TEXT)")
cur.execute("CREATE TABLE IF NOT EXISTS bookings (id INTEGER PRIMARY KEY, username TEXT,
campus TEXT, resource TEXT, date TEXT, start INTEGER, duration INTEGER)")
conn.commit()

cur.execute("SELECT * FROM users")
if len(cur.fetchall()) == 0:
    cur.execute("INSERT INTO users VALUES ('lecturer1', '1234', 'Lecturer')")
    cur.execute("INSERT INTO users VALUES ('admin1', '1234', 'CampusAdmin')")
    cur.execute("INSERT INTO users VALUES ('operator1', '1234', 'SystemOperator')")
    conn.commit()

campus_rules = {
    "Centurion": [8, 8, 17],
    "Pretoria": [7, 8, 18],
    "Polokwane": [9, 7, 18]
}

resources = {
    "Centurion": ["LAB1 - Computer Lab 1", "PROJ1 - Projector 1"],
    "Pretoria": ["LAB2 - Computer Lab 2", "PROJ2 - Projector 2"],
    "Polokwane": ["LAB3 - Computer Lab 3", "PROJ3 - Projector 3"]
}

class User:
    def __init__(self, name, password, role):
        self.name = name
        self.password = password
        self.role = role

class Lecturer(User):
    def __init__(self, name, password):
        User.__init__(self, name, password, "Lecturer")

class CampusAdmin(User):
    def __init__(self, name, password):
        User.__init__(self, name, password, "CampusAdmin")

class SystemOperator(User):
    def __init__(self, name, password):
        User.__init__(self, name, password, "SystemOperator")
```

```
current_user = None
current_campus = "Centurion"

window = tk.Tk()
window.title("Richfield - Smart Campus Booking System")
window.geometry("900x700")

login_frame = tk.LabelFrame(window, text="Login / Register")
login_frame.pack(fill="x", padx=10, pady=5)

tk.Label(login_frame, text="Username:").grid(row=0, column=0)
username_box = tk.Entry(login_frame)
username_box.grid(row=0, column=1)

tk.Label(login_frame, text="Password:").grid(row=1, column=0)
password_box = tk.Entry(login_frame, show="*")
password_box.grid(row=1, column=1)

tk.Label(login_frame, text="Role:").grid(row=2, column=0)
role_box = ttk.Combobox(login_frame, values=["Lecturer", "CampusAdmin", "SystemOperator"])
role_box.set("Lecturer")
role_box.grid(row=2, column=1)

status_label = tk.Label(login_frame, text="Not logged in", fg="red")
status_label.grid(row=0, column=2, padx=10)

def login():
    global current_user
    name = username_box.get()
    pw = password_box.get()
    role = role_box.get()

    cur.execute("SELECT * FROM users WHERE username = ?", (name,))
    row = cur.fetchone()

    if row == None:
        messagebox.showerror("Error", "User not found. Register first")
        return
    if row[1] != pw:
        messagebox.showerror("Error", "Wrong password")
        return
    if row[2] != role:
        messagebox.showerror("Error", "Your role is " + row[2])
        return

    if role == "Lecturer":
        current_user = Lecturer(name, pw)
```

```
    elif role == "CampusAdmin":
        current_user = CampusAdmin(name, pw)
    else:
        current_user = SystemOperator(name, pw)

    status_label.config(text="Logged in: " + name + " | Role: " + role, fg="blue")
    messagebox.showinfo("Success", "Welcome " + name)

def register():
    name = username_box.get()
    pw = password_box.get()
    role = role_box.get()

    if name == "" or pw == "":
        messagebox.showerror("Error", "Type username and password")
        return

    cur.execute("SELECT * FROM users WHERE username = ?", (name,))
    if cur.fetchone() != None:
        messagebox.showerror("Error", "Username already exists")
        return

    cur.execute("INSERT INTO users VALUES (?, ?, ?)", (name, pw, role))
    conn.commit()
    messagebox.showinfo("Success", "Registered! Now login")

tk.Button(login_frame, text="Login", command=login).grid(row=3, column=0)
tk.Button(login_frame, text="Register", command=register).grid(row=3, column=1)

campus_frame = tk.LabelFrame(window, text="Campus Selection")
campus_frame.pack(fill="x", padx=10, pady=5)

campus_box = ttk.Combobox(campus_frame, values=["Centurion", "Pretoria", "Polokwane"])
campus_box.set("Centurion")
campus_box.pack(side="left")

policy_label = tk.Label(campus_frame, text="Centurion: Max 2 hours, 08:00-17:00")
policy_label.pack(side="left", padx=10)

def select_campus():
    global current_campus
    current_campus = campus_box.get()
    rule = campus_rules[current_campus]
    policy_label.config(text=current_campus + ": Max " + str(rule[0]) + " hours, " + str(rule[1]) + ":00-"
+ str(rule[2]) + ":00")

    resource_list.delete(0, tk.END)
    for item in resources[current_campus]:
```

```
        resource_list.insert(tk.END, item)

tk.Button(campus_frame, text="Select Campus", command=select_campus).pack(side="left",
padx=10)

book_frame = tk.LabelFrame(window, text="Resources & Booking")
book_frame.pack(fill="x", padx=10, pady=5)

resource_list = tk.Listbox(book_frame, height=4)
resource_list.pack(fill="x")
resource_list.insert(tk.END, "LAB1 - Computer Lab 1")
resource_list.insert(tk.END, "PROJ1 - Projector 1")

tk.Label(book_frame, text="Date (YYYY-MM-DD):").pack()
date_box = tk.Entry(book_frame)
date_box.insert(0, "2026-10-22")
date_box.pack()

tk.Label(book_frame, text="Start Hour:").pack()
start_box = tk.Entry(book_frame)
start_box.insert(0, "9")
start_box.pack()

tk.Label(book_frame, text="Duration:").pack()
duration_box = tk.Entry(book_frame)
duration_box.insert(0, "1")
duration_box.pack()

def book():
    if current_user == None:
        messagebox.showerror("Error", "Login first")
        return

    selected = resource_list.curselection()
    if len(selected) == 0:
        messagebox.showerror("Error", "Select a resource")
        return

    full = resource_list.get(selected[0])
    resource_id = full.split(" - ")[0]
    date = date_box.get()
    start = int(start_box.get())
    duration = int(duration_box.get())

    rule = campus_rules[current_campus]
    max_hours = rule[0]
    open_time = rule[1]
    close_time = rule[2]
```

```
    if duration > max_hours:
        messagebox.showerror("Error", current_campus + " max is " + str(max_hours) + " hours")
        return

    if start < open_time or start + duration > close_time:
        messagebox.showerror("Error", "Outside campus hours")
        return

    # Check if already booked - simple loop
    cur.execute("SELECT start, duration FROM bookings WHERE campus=? AND resource=? AND
date=?", (current_campus, resource_id, date))
    rows = cur.fetchall()
    for old in rows:
        old_start = old[0]
        old_duration = old[1]
        if start < old_start + old_duration and old_start < start + duration:
            messagebox.showerror("Error", "Already booked! Choose another time")
            return

    cur.execute("INSERT INTO bookings (username, campus, resource, date, start, duration) VALUES (?,
?, ?, ?, ?, ?)", (current_user.name, current_campus, resource_id, date, start, duration))
    conn.commit()
    messagebox.showinfo("Success", "Booked " + resource_id + " on " + date)

tk.Button(book_frame, text="Book Selected Resource", bg="yellow", command=book).pack(pady=5)

def my_bookings():
    if current_user == None:
        messagebox.showerror("Error", "Login first")
        return
    cur.execute("SELECT * FROM bookings WHERE username=?", (current_user.name,))
    rows = cur.fetchall()
    if len(rows) == 0:
        messagebox.showinfo("My Bookings", "No bookings yet")
        return
    result = ""
    for r in rows:
        result = result + str(r) + "\n"
    messagebox.showinfo("My Bookings", result)

def report():
    cur.execute("SELECT campus, COUNT(*), AVG(duration) FROM bookings GROUP BY campus")
    rows = cur.fetchall()
    if len(rows) == 0:
        messagebox.showinfo("Report", "No bookings yet")
        return
    result = ""
```

```
    for r in rows:
        result = result + r[0] + ": Total=" + str(r[1]) + ", Avg=" + str(r[2]) + "\n"
    messagebox.showinfo("Report", result)

tk.Button(window, text="My Bookings", command=my_bookings).pack(side="left", padx=10)
tk.Button(window, text="Generate Report", command=report).pack(side="left", padx=10)

# Admin add resource
admin_frame = tk.LabelFrame(window, text="Add Resource (Admin Only)")
admin_frame.pack(fill="x", padx=10, pady=5)

tk.Label(admin_frame, text="ID:").pack(side="left")
new_id_box = tk.Entry(admin_frame, width=10)
new_id_box.pack(side="left")

tk.Label(admin_frame, text="Name:").pack(side="left")
new_name_box = tk.Entry(admin_frame, width=20)
new_name_box.pack(side="left")

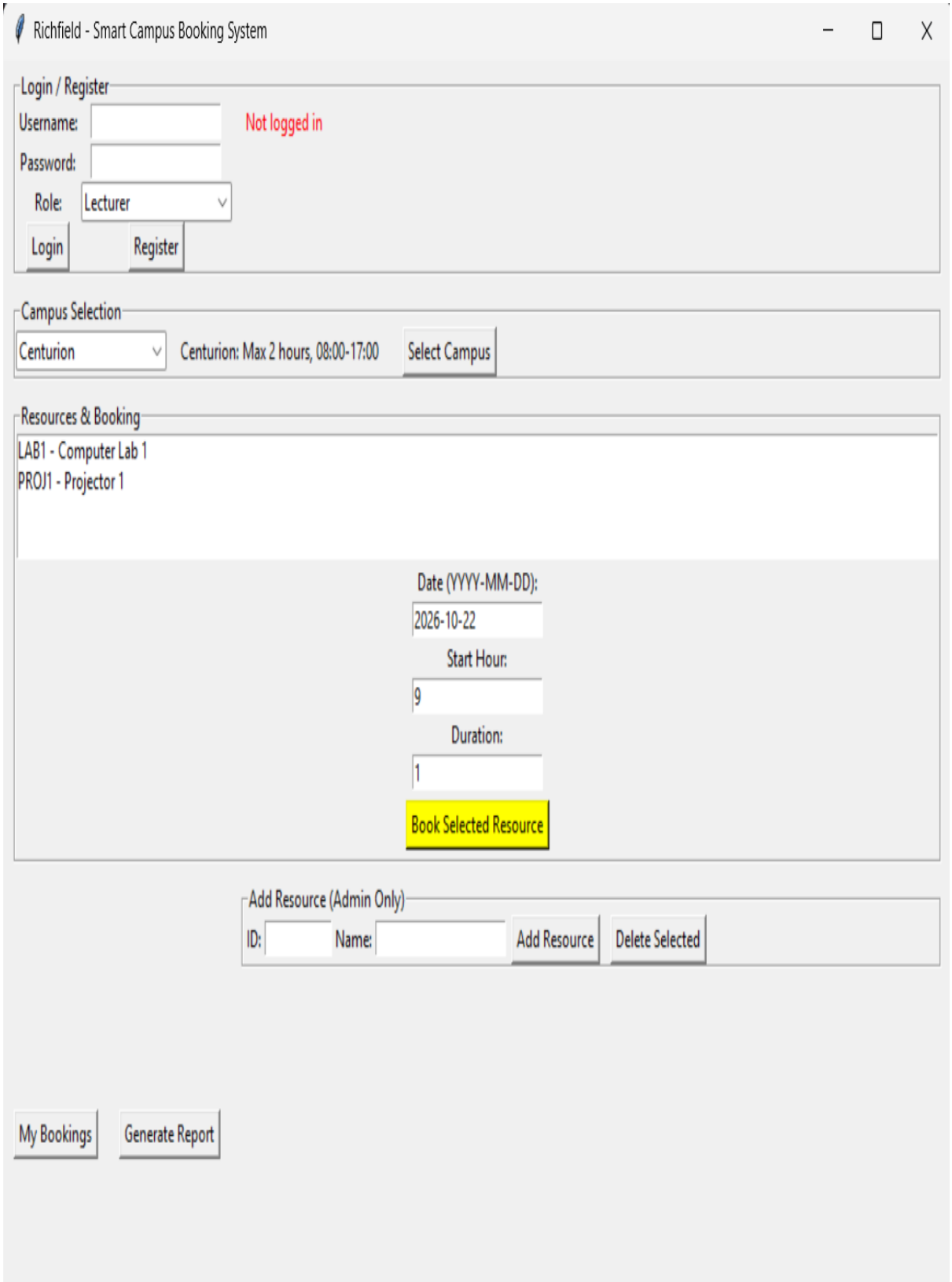
def add_resource():
    if current_user == None or current_user.role != "CampusAdmin":
        messagebox.showerror("Error", "Only Admin can add")
        return
    rid = new_id_box.get()
    name = new_name_box.get()
    if rid == "" or name == "":
        messagebox.showerror("Error", "Type ID and Name")
        return
    new_item = rid + " - " + name
    resources[current_campus].append(new_item)
    resource_list.insert(tk.END, new_item)
    messagebox.showinfo("Success", "Added " + new_item)

def delete_resource():
    if current_user == None or current_user.role != "CampusAdmin":
        messagebox.showerror("Error", "Only Admin can delete")
        return
    selected = resource_list.curselection()
    if len(selected) == 0:
        messagebox.showerror("Error", "Select resource to delete")
        return
    item = resource_list.get(selected[0])
    resources[current_campus].remove(item)
    resource_list.delete(selected[0])
    messagebox.showinfo("Success", "Deleted " + item)

tk.Button(admin_frame, text="Add Resource", command=add_resource).pack(side="left", padx=5)
tk.Button(admin_frame, text="Delete Selected", command=delete_resource).pack(side="left",
```

```
padx=5)

window.mainloop()
```


Richfield - Smart Campus Booking System
Login / Register
Username:
Not logged in
Password:
Role:
Lecturer
Login
Register
Campus Selection
Centurion
Centurion: Max 2 hours, 08:00-17:00
Select Campus
Resources & Booking
LAB1 - Computer Lab 1
PROJ1 - Projector 1
Date (YYYY-MM-DD):
2026-10-22
Start Hour:
9
Duration:
1
Book Selected Resource
Add Resource (Admin Only)
ID:
Name:
Add Resource
Delete Selected
My Bookings
Generate Report

Conclusion

The Smart Campus Resource Booking System was successfully developed using Python, Tkinter and SQLite. The system that I have created solves the problem of manual bookings at Richfield. It stops double bookings, it enforces campus rules automatically, and it saves all data in a database so that bookings are not lost when the program is closed. The system was tested with 3 campuses: Centurion, Pretoria and Polokwane.

In Centurion the system correctly blocked a booking and showed the error, in Pretoria the system correctly blocked a booking and showed the error, and in Polokwane the system correctly allowed a booking. The system also successfully tested login and register for Lecturer, Campus Admin and System Operator, booking a resource and preventing overlapping bookings for the same resource, cancelling a booking using the booking ID, adding a resource and detecting duplicate resource IDs, updating and deleting a resource by the Campus Administrator, generating a report that shows total bookings, average booking duration, and most frequently used resource for each campus, and cross-campus comparison by the System Operator.

Reference

GeeksforGeeks (2020b). *Python tkinter tutorial*. [online] GeeksforGeeks. Available at: https://www.geeksforgeeks.org/python/python-tkinter-tutorial/ [Accessed 22 Oct. 2026].

GeeksforGeeks (2020b). *Python Tkinter MessageBox Widget*. [online] GeeksforGeeks. Available at: https://www.geeksforgeeks.org/python/python-tkinter-messagebox-widget/ [Accessed 22 Oct. 2026].

GeeksforGeeks (2021b). *Python SQLite*. [online] GeeksforGeeks. Available at: https://www.geeksforgeeks.org/python/python-sqlite/ [Accessed 22 Oct. 2026].

GeeksforGeeks (2021b). *Python database tutorial*. [online] GeeksforGeeks. Available at: https://www.geeksforgeeks.org/python/python-database-tutorial/ [Accessed 22 Oct. 2026].

CodeGenes Team (2026). *Creating a database with python: A comprehensive guide*. [online] codegenes. Available at: https://www.codegenes.net/blog/how-to-create-a-database-with-python/ [Accessed 22 Oct. 2026].

GeeksforGeeks (2021c). *Python database tutorial*. [online] GeeksforGeeks. Available at: https://www.geeksforgeeks.org/python/python-database-tutorial/ [Accessed 22 Oct. 2026].

GeeksforGeeks (2018a). *Dictionaries in Python*. [online] GeeksforGeeks. Available at: https://www.geeksforgeeks.org/python/python-dictionary/ [Accessed 22 Oct. 2026].

Team, P. (2021). *Dictionaries in Python*. [online] Python Geeks. Available at: https://pythongeeks.org/dictionaries-in-python/ [Accessed 22 Oct. 2026].

GeeksforGeeks (2019). *Python classes and objects*. [online] GeeksforGeeks. Available at: https://www.geeksforgeeks.org/python/python-classes-and-objects/ [Accessed 22 Oct. 2026].